Assignment 6. jQuery

Team: FORYOU
Members: Asankhan Shyngys and Shangerey Akerke
Group: SE-2504

Objective

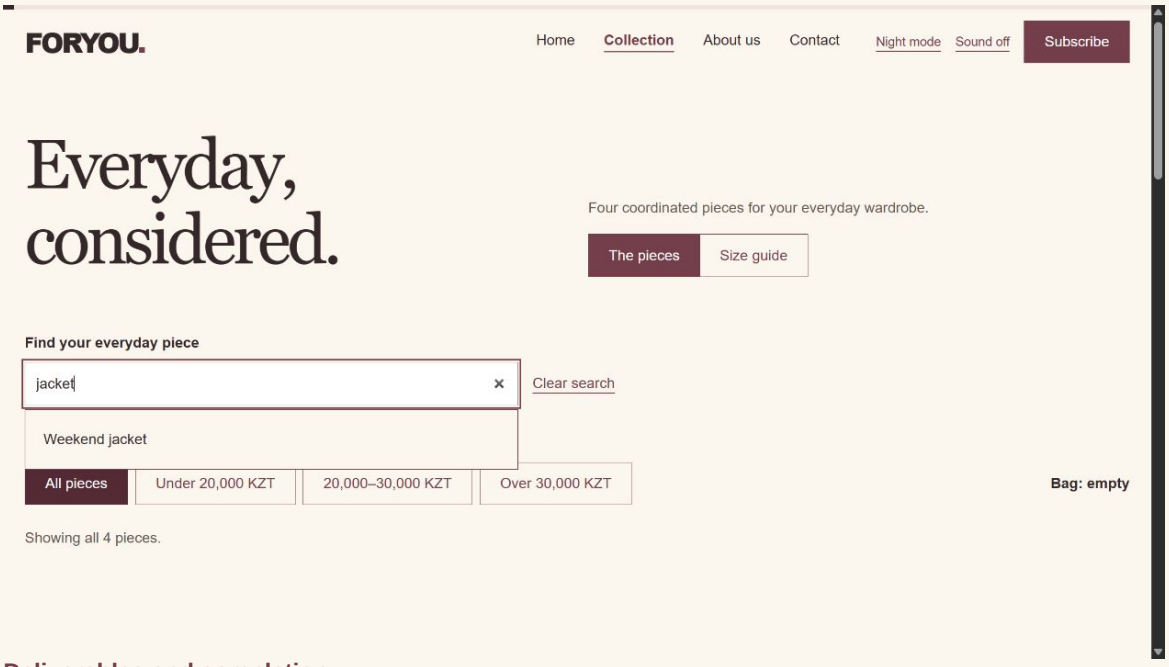
Continue the previous FORYOU project with jQuery to simplify DOM manipulation, handle events, improve search and add useful animation and interaction. Preserve the existing responsive clothing-store pages and earlier JavaScript features.

Deployed website

https://shanggereyakerke.github.io/FORYOU_Assignment_5/

Shared repository: github.com/ShanggereyAkerke/FORYOU_Assignment_5
Feature commit: 1a322416301aca0106f2c84d1818a59b2f907ee9

Deployment was verified: GitHub Pages build status **built**, deployment workflow **success**, live script HTTP **200**. On the deployed Collection page, searching jacket returned **1 piece** without console errors.



Deliverables and completion

Tasks 0-9 are implemented. Each task below includes the steps taken, a screenshot of actual project source rendered in a browser, a screenshot of the resulting webpage, and a checked result. The project ZIP contains HTML, CSS, JavaScript, local libraries, images, evidence and this report.

Both team members must upload the PDF and project ZIP to LMS before the published deadline and defend the work during practice. Uploading to LMS and the oral defense remain the students' responsibility.

Task 0. Setup and ready callback

1. Add the required jQuery 4.0.0 CDN script in the head of each main page, followed by the external script.js file.

2. Keep the same library locally and load it if the CDN fails. Initialize features inside $(document).ready().

3. Open the browser console and verify the exact message: jQuery is ready!

Verified: The captured browser log contains the exact ready message; the four main pages load the new UI without JavaScript errors.

Code screenshot | index.html

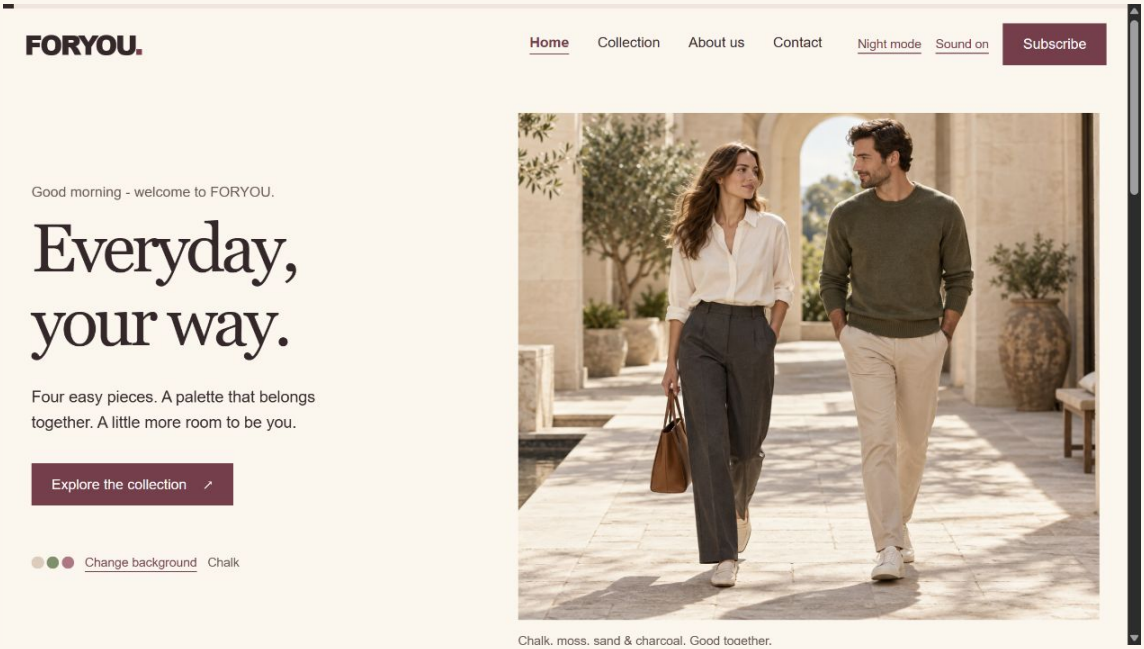
Task 0 | index.html | source lines 12-14

```
 12    <script src="https://code.jquery.com/jquery-4.0.0.min.js"></script>
 13    <script>if (!window.jQuery) document.write('<script src="vendor/jquery-4.0.0.min.js"><\/script>');</script>
 14    <script src="script.js?v=6.1" defer></script>

script.js
/* Assignment 6: jQuery search, engagement and useful browser interactions. */
$(document).ready(function () {
  "use strict";
  console.log("jQuery is ready!"); // Task 0
```

Actual source excerpt from the FORYOU project, rendered for screenshot evidence.

Resulting webpage



Task 1. Real-time search and live filter

1. Add a labelled Collection search input, a live result count, a Clear search button and an empty state.

2. Listen with the jQuery input event so typing, pasting and mobile input all update the filter immediately.

3. Compare lowercase product titles, descriptions and alternative text; hide nonmatching columns. Reapply search whenever price filtering rerenders products.

Verified: shirt gives one piece; [gives zero without a regex error; knit stays active across price changes and returns when All pieces is selected.

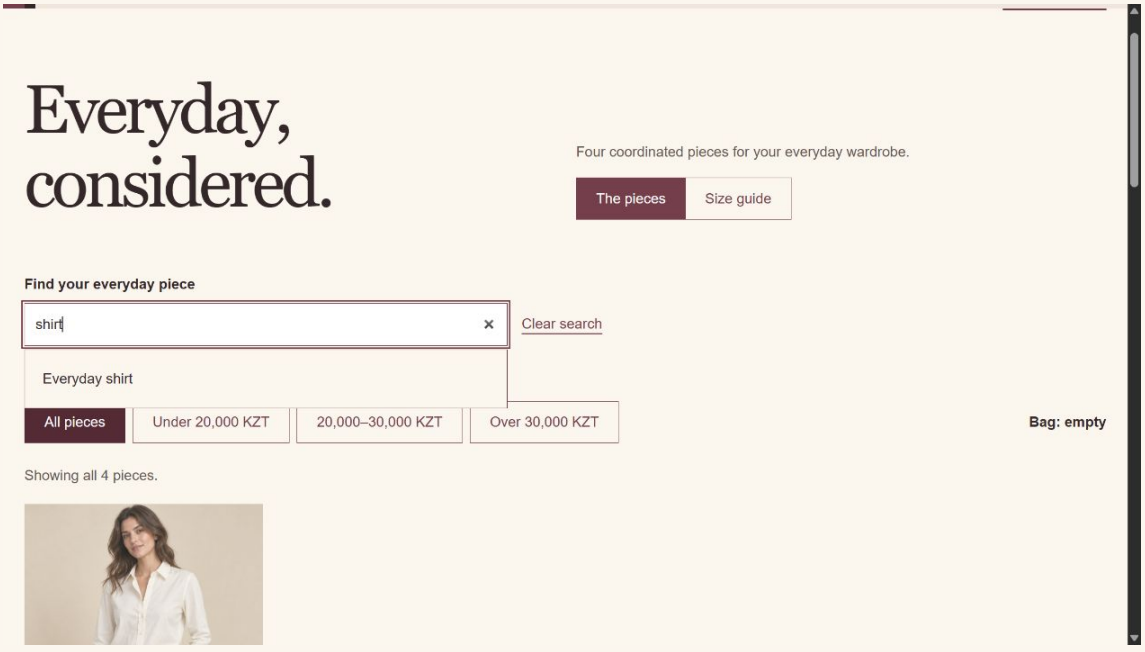
Code screenshot | script.js

Task 1 | script.js | source lines 154-165

```
154   function filterProducts(showSuggestions) {
155     const keyword = String($search.val() || "").trim();
156     const query = keyword.toLocaleLowerCase();
157     let count = 0;
158     $("#product-row .product-card").each(function () {
159       const $card = $(this);
160       const text = $card.find(".card-title, .card-text").text() + " " + ($card.find("img").attr("alt") || "");
161       const matches = text.toLocaleLowerCase().includes(query);
162       $card.parent().prop("hidden", !matches);
163       if (matches) count++;
164       highlight(this, keyword);
165     });
```

Actual source excerpt from the FORYOU project, rendered for screenshot evidence.

Resulting webpage



Task 2. Autocomplete search suggestions

1. Read product titles into a suggestion list and add matching titles as listbox options with jQuery.

2. Track the active option using aria-selected and aria-activedescendant while keeping focus in the input.

3. Support mouse selection, Up/Down arrows and Enter. Dismiss on Escape, Tab, blur or an empty search.

Verified: Typing sh offers Everyday shirt. ArrowDown and Enter select it and close the dropdown; the product remains filtered.

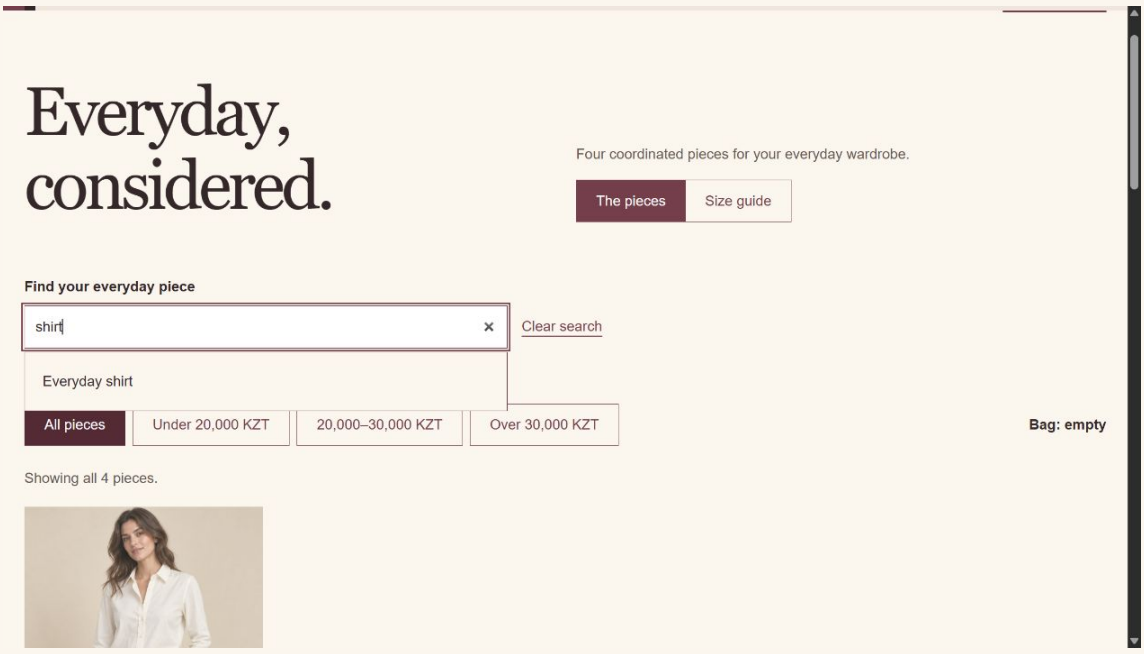
Code screenshot | script.js

Task 2 | script.js | source lines 181-192

```
181    $search.on("keydown", function (event) {
182      const $options = $suggestions.children();
183      if (event.key === "Escape") { closeSuggestions(); return; }
184      if (!$options.length) return;
185      if (event.key === "ArrowDown" || event.key === "ArrowUp") {
186        event.preventDefault();
187        selection = event.key === "ArrowDown" ? (selection + 1) % $options.length : (selection < 0 ? $options.length
- 1 : (selection - 1 + $options.length) % $options.length);
188        $options.attr("aria-selected", "false").eq(selection).attr("aria-selected", "true");
189        $search.attr("aria-activedescendant", $options.eq(selection).attr("id"));
190      } else if (event.key === "Enter" && selection >= 0) {
191        event.preventDefault();
192        $search.val($options.eq(selection).text());
```

Actual source excerpt from the FORYOU project, rendered for screenshot evidence.

Resulting webpage



Task 3. Search highlighting

1. Remove previous marks and normalize adjacent text nodes before every new search.

2. Escape regex metacharacters so the keyword is treated literally. Walk text nodes and wrap every occurrence in a mark element.

3. Apply highlights to searchable product content and FAQ questions/answers; reveal matching FAQ answers while preserving their existing links.

Verified: Searching size highlights occurrences in both the FAQ heading and answer. The count is one question; clearing the input removes the marks.

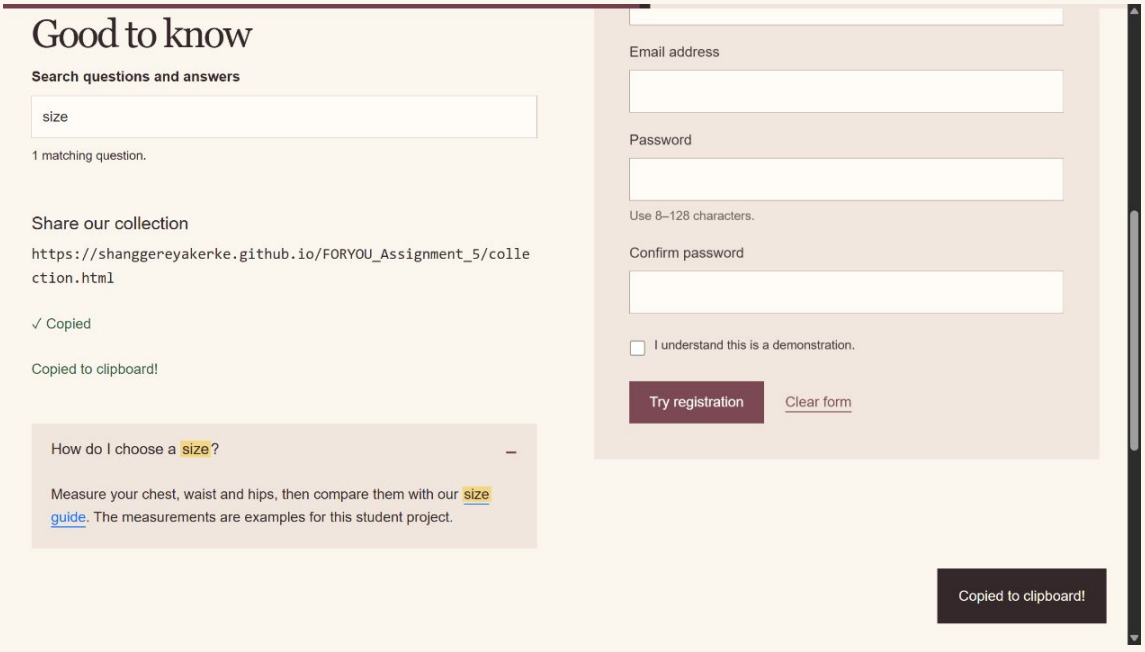
Code screenshot | script.js

Task 3 | script.js | source lines 117-128

```
117    function highlight(root, keyword) {
118      $(root).find("mark.search-match").each(function () {
this.replaceWith(document.createTextNode(this.textContent)); });
119      root.normalize();
120      if (!keyword) return;
121      const escaped = keyword.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
122      const regex = new RegExp(escaped, "gi");
123      const walker = document.createTreeWalker(root, NodeFilter.SHOW_TEXT);
124      const nodes = [];
125      while (walker.nextNode()) nodes.push(walker.currentNode);
126      nodes.forEach(function (node) {
127        if (node.parentElement.closest("script, style, mark, .rating, .rating-text, [data-add]")) return;
128        const content = node.textContent;
```

Actual source excerpt from the FORYOU project, rendered for screenshot evidence.

Resulting webpage



Task 4. Stylized scroll progress bar

1. Prepend an accessible fixed progress track using the existing rose and ink colors with a contrasting leading marker.

2. Calculate the percentage as scrollTop divided by document height minus viewport height. Clamp the result to 0-100.

3. Use jQuery scroll, resize and load listeners to update width and aria-valuenow; recalculate after search and image loading.

Verified: Scrolling to the bottom of About reports aria-valuenow=100; the filled top bar is visible in the screenshot.

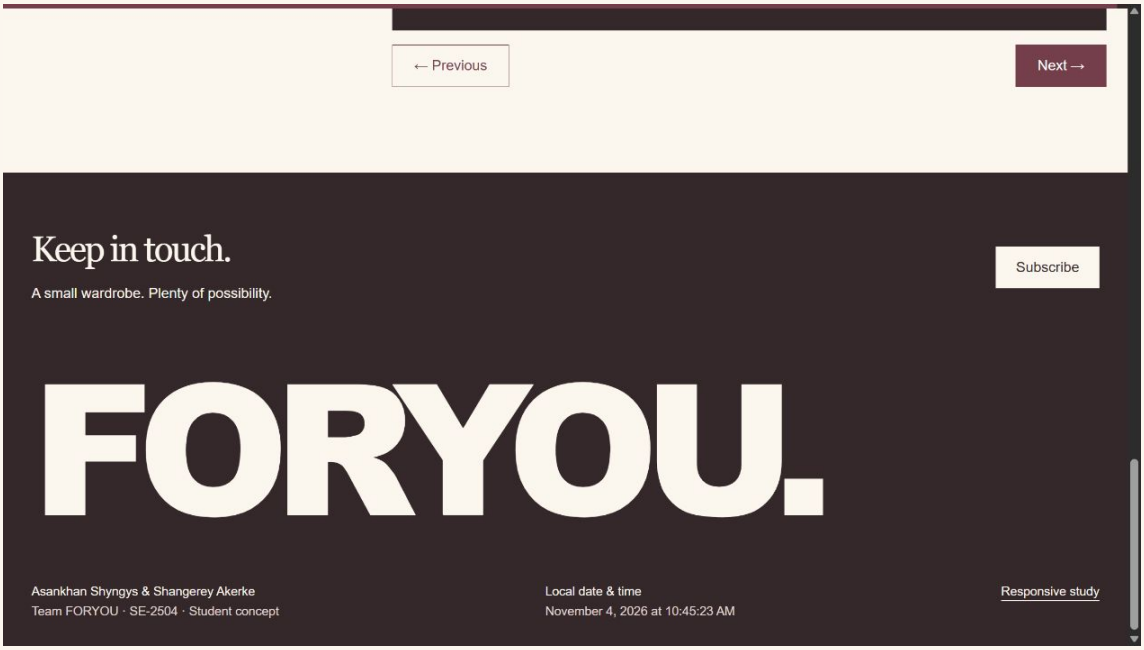
Code screenshot | script.js

Task 4 | script.js | source lines 52-60

```
52    const $progress = $('<div class="reading-progress" role="progressbar" aria-label="Page scroll progress" aria-
valuemin="0" aria-valuemax="100" aria-valuenow="0"><span></span></div>').prependTo("body");
53    function updateProgress() {
54      const distance = Math.max(0, $(document).height() - $(window).height());
55      const percent = distance ? Math.min(100, Math.max(0, $(window).scrollTop() / distance * 100)) : 100;
56      $progress.attr("aria-valuenow", Math.round(percent)).find("span").css("width", percent + "%");
57    }
58    $(window).on("scroll.assignment6 resize.assignment6 load.assignment6", updateProgress);
59    updateProgress();
60
```

Actual source excerpt from the FORYOU project, rendered for screenshot evidence.

Resulting webpage



Task 5. Animated number counter

1. Add real project statistics on About: four everyday pieces, nine lookbook images and two team members.

2. Check viewport visibility on scroll and resize, and mark each counter as started so it animates only once.

3. Use jQuery animate on a numeric object for a 1.6-second count-up and set the exact final value. Respect reduced-motion preferences.

Verified: All three counters finish at 4, 9 and 2. Their numbers are visible without JavaScript as a readable fallback.

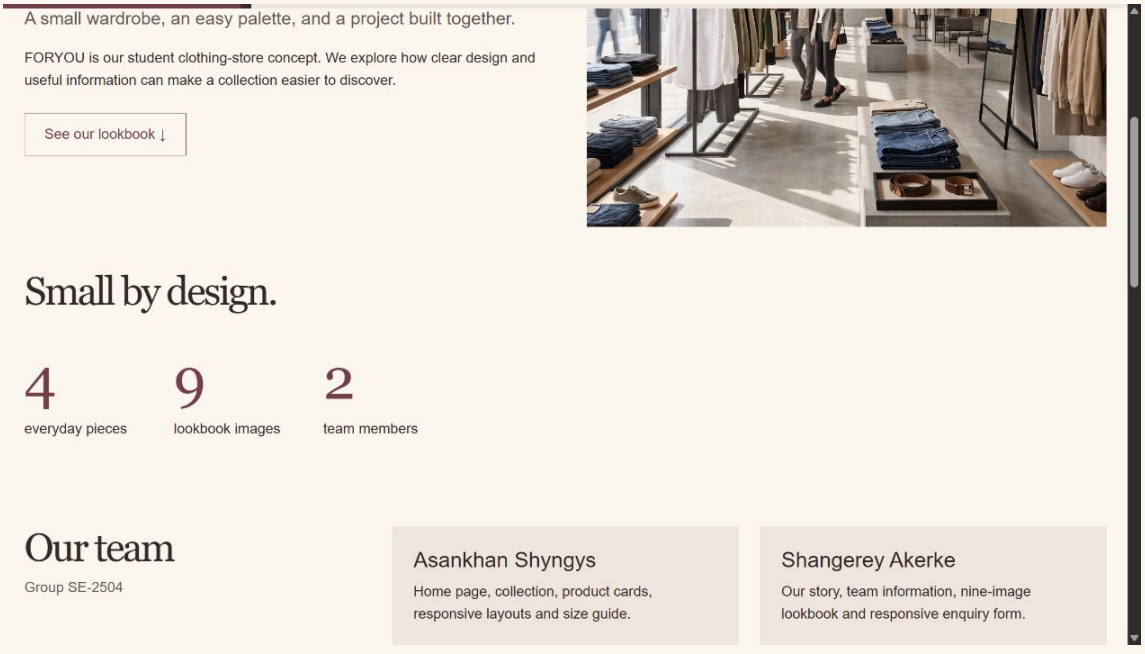
Code screenshot | script.js

Task 5 | script.js | source lines 62-73

```
62  function animateCounter(element) {
63    const $element = $(element);
64    if ($element.data("counted")) return;
65    $element.data("counted", true);
66    const target = Number($element.attr("data-count"));
67    if (reduceMotion) { $element.text(target); return; }
68    $({ count: 0 }).animate({ count: target }, {
69      duration: 1600,
70      step: function (value) { $element.text(Math.floor(value)); },
71      complete: function () { $element.text(target); }
72    });
73  }
```

Actual source excerpt from the FORYOU project, rendered for screenshot evidence.

Resulting webpage



Task 6. Loading spinner on submit

1. Retain existing validation; only a valid registration or subscription invokes the shared loading helper.

2. Disable the submit button, display a CSS spinner and Please wait text, and mark the form aria-busy. Ignore repeated pending submits.

3. After 1.4 seconds, restore the button and display the demo result. Resetting the form or closing the dialog cancels pending work.

Verified: Subscription showed a disabled loading button, then restored Try subscribing with its validation result. Invalid registration did not start loading; Clear form canceled a valid pending registration.

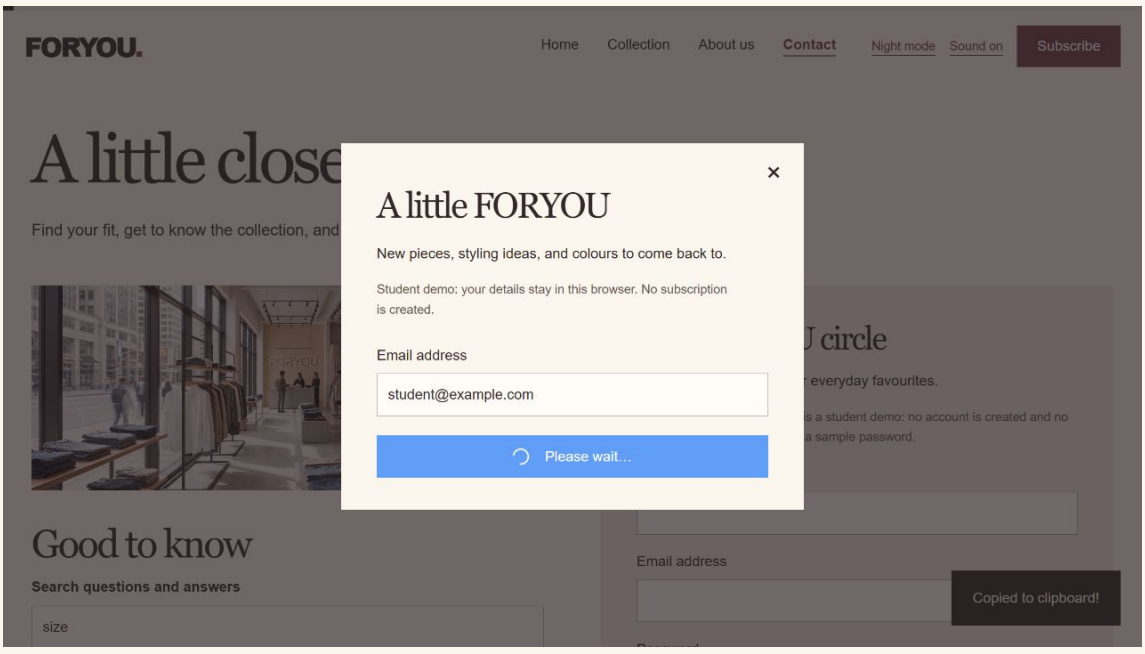
Code screenshot | script.js

Task 6 | script.js | source lines 20-31

```
20    function submit(form, complete) {
21      if (pending.has(form)) return;
22      const $button = $(form).find('[type="submit"]');
23      const original = $button.html();
24      $button.prop("disabled", true).html('<span class="submit-spinner" aria-hidden="true"></span> Please wait…');
25      $(form).attr("aria-busy", "true");
26      const restore = function () {
27        $button.prop("disabled", false).html(original);
28        $(form).removeAttr("aria-busy");
29        pending.delete(form);
30      };
31      const timer = setTimeout(function () {
```

Actual source excerpt from the FORYOU project, rendered for screenshot evidence.

Resulting webpage



Task 7. Temporary notification system

1. Create one shared role=status live region and expose a notify helper.

2. Trigger notifications after adding a product to the bag, a completed valid form simulation, or a successful copy.

3. Cancel the previous timer and animation when a new message arrives. Hold for 3.2 seconds and fade out with jQuery; reduce motion when requested.

Verified: Adding Everyday shirt shows Everyday shirt added to your bag. Notifications disappear rather than accumulating.

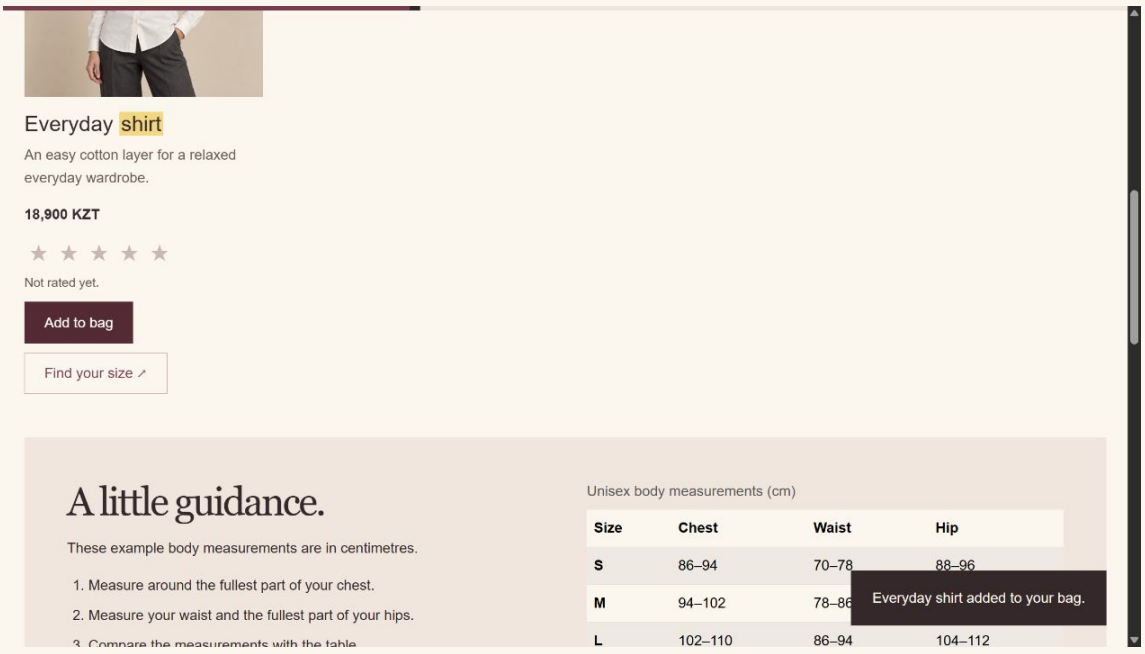
Code screenshot | script.js

Task 7 | script.js | source lines 8-17

```
8   const $toast = $('<div class="foryou-toast" role="status" aria-live="polite" hidden></div>').appendTo("body");
9   let toastTimer;
10  function notify(message) {
11    clearTimeout(toastTimer);
12    $toast.stop(true, true).text(message).prop("hidden", false).show();
13    toastTimer = setTimeout(function () {
14      $toast.fadeOut(reduceMotion ? 0 : 350, function () { $toast.prop("hidden", true); });
15    }, 3200);
16  }
17
```

Actual source excerpt from the FORYOU project, rendered for screenshot evidence.

Resulting webpage



Task 8. Copy to clipboard button

1. Place a Copy link control beside the collection URL on Contact, connected to a tooltip.

2. Handle the click with jQuery and await navigator.clipboard.writeText on localhost or HTTPS.

3. Only after success, change the icon to a checkmark, show Copied to clipboard in a tooltip and toast, then restore the control. Show a truthful manual-copy message on failure.

Verified: The browser clipboard was read back and exactly matched the public collection URL. The checkmark and success tooltip appeared.

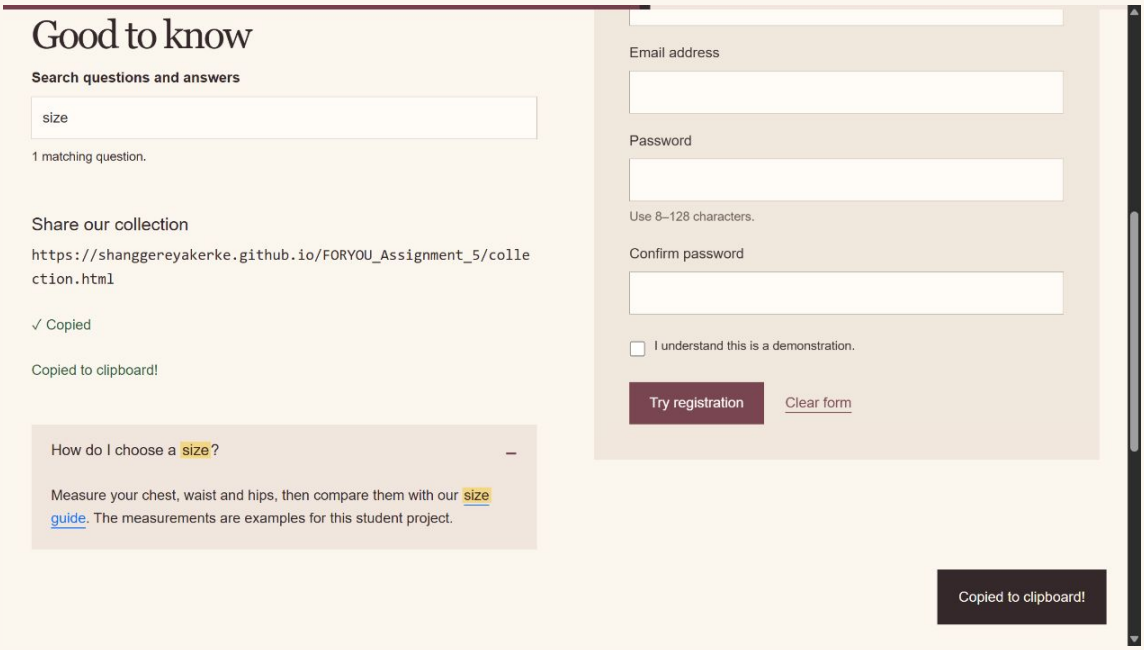
Code screenshot | script.js

Task 8 | script.js | source lines 98-109

```
 98   $("[data-copy-target]").on("click", async function () {
 99     const $button = $(this);
100     if ($button.prop("disabled")) return;
101     const original = $button.html();
102     const text = $(this.getAttribute("data-copy-target")).text().trim();
103     try {
104       if (!navigator.clipboard || !window.isSecureContext) throw new Error("Clipboard unavailable");
105       await navigator.clipboard.writeText(text);
106       $button.prop("disabled", true).html('<span aria-hidden="true">✓</span> Copied'); // taste-skill-ignore:
assignment requires a checkmark
107       const $tip = $button.siblings('[role="tooltip"]');
108       $tip.text("Copied to clipboard!").prop("hidden", false);
109       notify("Copied to clipboard!");
```

Actual source excerpt from the FORYOU project, rendered for screenshot evidence.

Resulting webpage



Task 9. Image lazy loading

1. Keep the main hero imagery eager. Move deferred image URLs into data-src and retain dimensions and static noscript fallbacks.

2. On jQuery scroll, resize and load events, test the image rectangle against the viewport and assign src only once it is visible.

3. For the hidden Bootstrap carousel, load the selected destination image on slide.bs.carousel just before its transition. Recheck newly rendered product cards after filtering.

Verified: About initially had nine deferred carousel images. Selecting Next assigned images/knit.png to the second slide while unvisited hidden slides remained deferred.

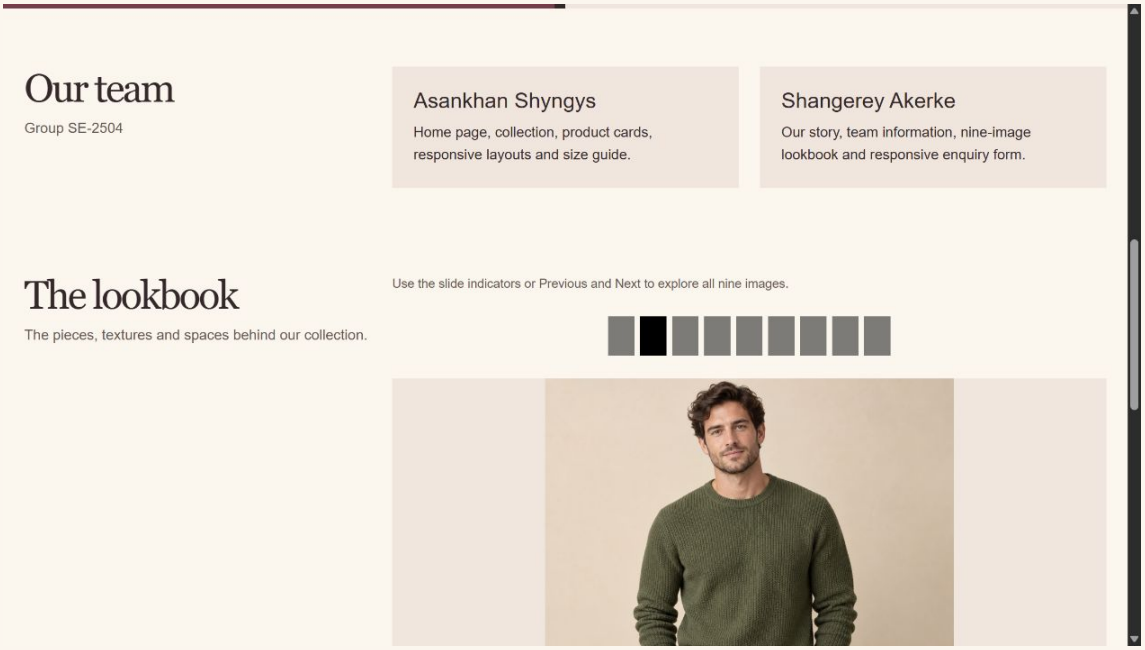
Code screenshot | script.js

Task 9 | script.js | source lines 83-94

```
83    function loadImage(image) {
84      const $image = $(image);
85      const source = $image.attr("data-src");
86      if (!source) return;
87      $image.one("load", function () { $(this).addClass("is-loaded"); updateProgress(); })
88        .one("error", function () { $(this).addClass("image-error"); });
89      $image.attr("src", source).removeAttr("data-src");
90    }
91    function checkImages() { $("img[data-src]").each(function () { if (inView(this)) loadImage(this); }); }
92    $(window).on("scroll.lazy resize.lazy load.lazy", checkImages);
93    // Bootstrap's next slide starts hidden; load it just before its transition.
94    $(".carousel").on("slide.bs.carousel", function (event) { $(event.relatedTarget).find("img[data-
src]").each(function () { loadImage(this); }); });
```

Actual source excerpt from the FORYOU project, rendered for screenshot evidence.

Resulting webpage



Verification and reflection

Browser verification

- Task 0: exact jQuery is ready! message captured; no console errors in tested feature flows.

- Search: live typing, suggestion selection with arrows and Enter, clearing, literal [input, no-result feedback, and price-filter rerenders.

- FAQ: matching questions/answers highlighted, answer revealed, existing size-guide link preserved.

- Forms: invalid submission rejected, valid submit disabled during spinner, completion restores button, reset cancels a pending operation.

- Clipboard: success tooltip and checkmark; copied text read back and matched the public URL.

- About: final values 4/9/2, deferred carousel image source assigned before transition, scroll progress reached 100%.

- Responsive: Collection, Contact and About checked at 390px with no horizontal overflow. Existing arrow-key header navigation still works.

- Static checks: four custom JavaScript files passed node --check; git diff --check passed; design source lint passed.

Challenges and solutions

Search and price filtering: the old collection renderer replaces the cards. It now emits a jQuery event after rendering so the current query is reapplied and newly visible images load.

Safe highlighting: user text may contain regex syntax or HTML. Escape regex symbols and create text/mark nodes instead of interpolating user input into HTML. A small FAQ layout adjustment keeps highlighted text flowing naturally.

Timers and carousel visibility: reset/close cancels pending submission timers. Hidden carousel slides do not intersect the viewport, so their selected destination image loads on the Bootstrap slide event.

Stable verification: screenshot output triggered the editor's live reload, so testing moved to a separate static server. The standalone CDP design inspector timed out; connected-browser screenshots and responsive measurements were used instead.

Reflection

jQuery reduces repeated DOM/event code, but correctness still depends on lifecycle and state: rerenders, focus, async success, timers and hidden elements need explicit handling. Real project statistics and honest demo messages avoid claiming customers, subscriptions or orders that do not exist.

Defense and submission checklist

Suggested defense split

Asankhan Shyngys: Collection live search, autocomplete, product highlighting, scroll calculation, bag notification event and product lazy loading.
Shangerey Akerke: FAQ highlighting, About counters, validated loading states, clipboard feedback, toast animation and carousel loading.
Both members: understand all code, test the deployed site, explain integration with prior assignments, and submit individually.

Questions to be ready to answer

Why input rather than only keyup? input covers typing, paste, deletion and mobile input.

Why $(document).ready()? It initializes handlers after the document and existing renderers are ready.

How do suggestions keep focus? The active option is tracked with ARIA attributes while the combobox retains keyboard focus.

Why escape the keyword? Regex metacharacters must be treated as literal user text; text nodes prevent injected HTML.

How is scroll progress computed? scrollTop / (documentHeight - viewportHeight) times 100, clamped to the supported range.

How is repeated submit prevented? Disable the button and keep one pending timer per form; reset or dialog close cancels it.

Why await the clipboard promise? The checkmark must represent a successful copy rather than an attempted operation.

Why does the carousel need a special event? Hidden slides have no visible rectangle. Load the selected destination before its transition.

Final student actions

1. Open the public URL and rehearse each demonstration.
2. Each member submits the PDF report and complete project ZIP in LMS.
3. Check the deadline shown in LMS.
4. Attend the practice defense and be ready to modify the code as requested by the instructor.

Source brief: Assignment 6 jQuery.pdf supplied by the user. Dependencies: jQuery 4.0.0 (OpenJS Foundation) and the existing Bootstrap bundle. No data is submitted to a backend by the demonstration forms.